МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ №2

з дисципліни **"Функціональне програмування"**

на тему **"ФУНКЦІЇ ВИЩОГО ПОРЯДКУ, LAMBDA, КОМПОЗИЦІЯ"**

Варіант 14. Музичний каталог

Виконала:
Студентка групи ФЕП-23
Поведа Анастасія

Перевірив:
проф. Свелеба С. А.

Львів-2026

Мета: навчитися застосовувати функції вищого порядку, `lambda`, `map` / `filter` / `reduce` і композицію функцій; зібрати конвеєр перетворень над записами музичного каталогу без мутації вхідних даних та окремо виконати reduce-агрегацію.

Теоретичні відомості:

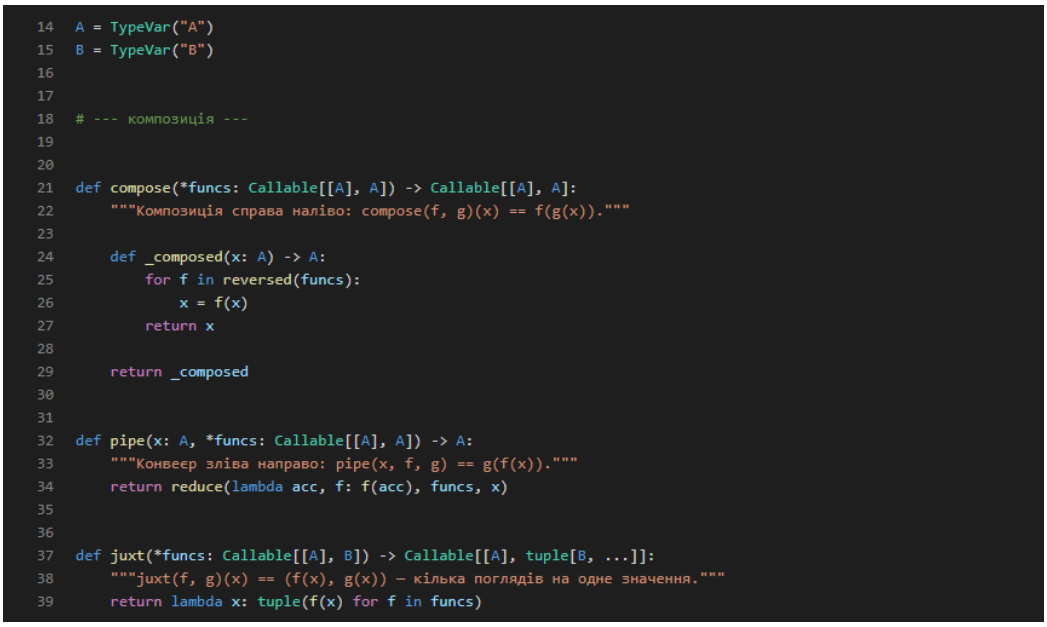
Функцією вищого порядку називають функцію, що отримує інші функції як аргументи або повертає функцію як результат. Оскільки в Python функції є звичайними значеннями, їх можна передавати, повертати з фабрик і зберігати у структурах даних. Найуживаніші функції вищого порядку — `map` для перетворення кожного елемента, `filter` для відбору за предикатом і `functools.reduce` для згортання послідовності в одне значення. Генераторні вирази є їхнім лінивим еквівалентом: елементи обчислюються лише тоді, коли їх споживають.

Композиція дає змогу описувати конвеєр як одну функцію без проміжних змінних. `compose(f, g)(x)` дорівнює `f(g(x))` і застосовує функції справа наліво, `pipe(x, f, g)` проганяє значення зліва направо й одразу повертає результат, а `juxt(f, g)(x)` повертає кортеж результатів кількох функцій для одного аргументу. Якщо кожен крок має однаковий тип `Iterable[Track] -> Iterable[Track]` і будує нові словники через `{**t, "поле": значення}`, конвеєр лишається чистим: вхідний список не змінюється, результат визначається тільки аргументами, а кроки можна перевіряти незалежно один від одного.

Хід роботи:

1. У `core.py` реалізувала засоби композиції: `compose` (справа наліво, повертає нову функцію), `pipe` (зліва направо через `reduce`) і `juxt` (кортеж результатів

кількох функцій). Типи описані через `TypeVar`, тому конвеєр з однотипних кроків проходить `mypy --strict`:



```python
A = TypeVar("A")
B = TypeVar("B")


# --- композиція ---


def compose(*funcs: Callable[[A], A]) -> Callable[[A], A]:
    """Композиція справа наліво: compose(f, g)(x) == f(g(x))."""

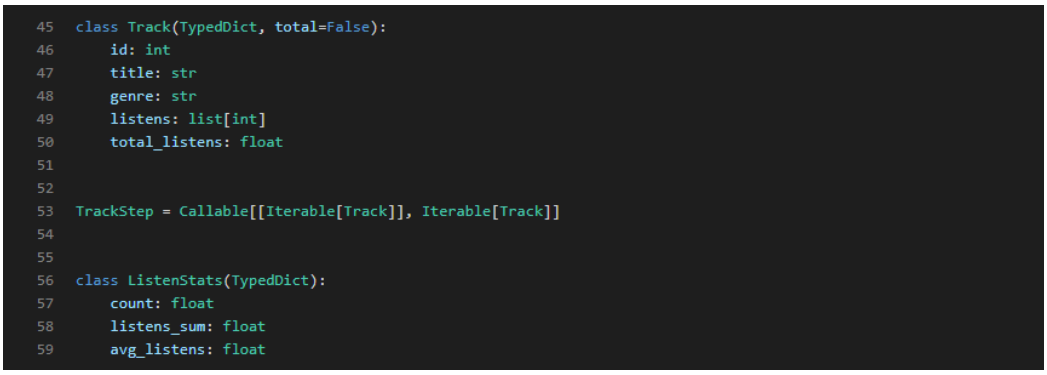
    def _composed(x: A) -> A:
        for f in reversed(funcs):
            x = f(x)
        return x

    return _composed


def pipe(x: A, *funcs: Callable[[A], A]) -> A:
    """Конвеєр зліва направо: pipe(x, f, g) == g(f(x))."""
    return reduce(lambda acc, f: f(acc), funcs, x)


def juxt(*funcs: Callable[[A], B]) -> Callable[[A], tuple[B, ...]]:
    """juxt(f, g)(x) == (f(x), g(x)) — кілька поглядів на одне значення."""
    return lambda x: tuple(f(x) for f in funcs)
```

2. За варіантом 14 описала запис треку `Track` через `TypedDict` (`id`, `title`, `genre`, список прослуховувань за періодами `listens` і обчислюване поле `total_listens`), тип кроку конвеєра `TrackStep` та структуру результату агрегації `ListenStats`:



```python
class Track(TypedDict, total=False):
    id: int
    title: str
    genre: str
    listens: list[int]
    total_listens: float


TrackStep = Callable[[Iterable[Track]], Iterable[Track]]


class ListenStats(TypedDict):
    count: float
    listens_sum: float
    avg_listens: float
```

3. Реалізувала чисті трансформації як фабрики, що повертають функції над `Iterable[Track]`: `with_total_listens` (map: сумарні прослуховування), `boost_genre` (map: підсилення лише для вибраного жанру), `min_listens` (filter із `lambda`-предикатом, що відкидає непопулярні треки), `normalize_titles` (map з `lambda`), `sort_by_listens_desc` (сортування з двома ключами: прослуховування

спадно, `id` зростом) і `take` (рейтинг Top-N через `itertools.islice`). Кожен крок повертає нові словники:

```python
def with_total_listens() -> TrackStep:
    """map: total_listens = сума прослуховувань за періодами (новий запис)."""
    return lambda tracks: (
        {**t, "total_listens": float(sum(t.get("listens", []) or []))} for t in tracks
    )


def boost_genre(genre: str, factor: float) -> TrackStep:
    """map: підсилити прослуховування лише для вибраного жанру."""

    def _boost(tracks: Iterable[Track]) -> Iterator[Track]:
        for t in tracks:
            total = float(t.get("total_listens", 0.0))
            boosted = total * factor if t.get("genre") == genre else total
            yield {**t, "total_listens": boosted}

    return _boost


def min_listens(threshold: float) -> TrackStep:
    """filter: відкинути непопулярні треки (нижче порога прослуховувань)."""
    return lambda tracks: filter(
        lambda t: float(t.get("total_listens", 0.0)) >= threshold, tracks
    )


def normalize_titles() -> TrackStep:
    """map: назва треку у Title Case без зайвих пробілів."""
    return lambda tracks: map(
        lambda t: {**t, "title": str(t.get("title", "")).strip().title()}, tracks
    )


def sort_by_listens_desc() -> TrackStep:
    """sorted з двома ключами: прослуховування спадно, id зростом."""
    return lambda tracks: sorted(
        tracks,
        key=lambda t: (-float(t.get("total_listens", 0.0)), int(t.get("id", 0))),
    )


def take(n: int) -> TrackStep:
    """Top-N (рейтинг) через itertools.islice."""
    return lambda tracks: list(islice(tracks, n))
```

4. Додала reduce-агрегацію `reduce_listen_stats` з трьома показниками (`count`, `listens_sum`, `avg_listens`) — акумулятором є кортеж, тому проміжний стан не мутується, — і функцію `build_chart_pipeline`, що збирає весь конвеєр через

`compose` і повертає чисту функцію з явними параметрами (Top-N, жанр, коефіцієнт, поріг):



```python
def reduce_listen_stats(tracks: Iterable[Track]) -> ListenStats:
    """reduce-агрегація: count, listens_sum, avg_listens."""
    count, total = reduce(
        lambda acc, t: (acc[0] + 1, acc[1] + float(t.get("total_listens", 0.0))),
        tracks,
        (0.0, 0.0),
    )
    return {
        "count": count,
        "listens_sum": total,
        "avg_listens": total / count if count else 0.0,
    }


def build_chart_pipeline(
    top_n: int = 3,
    genre: str = "rock",
    factor: float = 1.5,
    threshold: float = 0.0,
) -> Callable[[Iterable[Track]], list[Track]]:
    """Складає конвеєр варіанта 14 через compose; повертає чисту функцію."""
    steps = compose(
        take(top_n),
        sort_by_listens_desc(),
        min_listens(threshold),
        boost_genre(genre, factor),
        with_total_listens(),
        normalize_titles(),
    )
    return lambda tracks: list(steps(tracks))
```

5. Базовий приклад з методички (клієнти інтернет-магазину) також реалізувала у тому ж стилі: `only_adults`, `with_total`, `boost_city`, `normalize_names`,

`sort_by_total_desc`, `take_records`, `reduce_stats` і `build_pipeline`:

```python
class Record(TypedDict, total=False):
    id: int
    name: str
    age: int
    city: str
    purchases: list[float]
    total: float


RecordStep = Callable[[Iterable[Record]], Iterable[Record]]


class Stats(TypedDict):
    count: float
    sum_total: float
    avg_total: float


def only_adults(min_age: int = 18) -> RecordStep:
    """filter: повнолітні."""
    return lambda recs: (r for r in recs if int(r.get("age", 0)) >= min_age)


def with_total() -> RecordStep:
    """map: total = sum(purchases)."""
    return lambda recs: (
        {**r, "total": float(sum(r.get("purchases", []) or []))} for r in recs
    )


def boost_city(city: str, factor: float) -> RecordStep:
    """map: підсилити total для заданого міста."""

    def _boost(recs: Iterable[Record]) -> Iterator[Record]:
        for r in recs:
            t = float(r.get("total", 0.0))
            yield {**r, "total": t * factor if r.get("city") == city else t}

    return _boost


def normalize_names() -> RecordStep:
    """map: ім'я у Title Case."""
    return lambda recs: (
        {**r, "name": str(r.get("name", "")).strip().title()} for r in recs
    )


def sort_by_total_desc() -> RecordStep:
    """sorted: total спадно, id зростом."""
    return lambda recs: sorted(
        recs, key=lambda r: (-float(r.get("total", 0.0)), r.get("id", 0))
    )


def take_records(n: int) -> RecordStep:
    """Top-N для записів клієнтів."""
    return lambda recs: list(islice(recs, n))
```

```python
def reduce_stats(recs: Iterable[Record]) -> Stats:
    """reduce-агрегація: count, sum_total, avg_total."""
    count, total = reduce(
        lambda acc, r: (acc[0] + 1, acc[1] + float(r.get("total", 0.0))),
        recs,
        (0.0, 0.0),
    )
    return {
        "count": count,
        "sum_total": total,
        "avg_total": total / count if count else 0.0,
    }


def build_pipeline(
    top_n: int = 2, city: str = "Delhi", factor: float = 1.1
) -> Callable[[Iterable[Record]], list[Record]]:
    """Конвеєр базового прикладу з методички."""
    steps = compose(
        take_records(top_n),
        sort_by_total_desc(),
        boost_city(city, factor),
        with_total(),
        only_adults(18),
        normalize_names(),
    )
    return lambda recs: list(steps(recs))
```

6. Увесь I/O винесла в `app.py`: читання JSON, розбір аргументів командного рядка через `argparse` (`--data`, `--top`, `--boost-genre`, `--factor`, `--min-listens`), функцію `render`, яка лише перетворює чистий результат на текст, і друк:

```python
def load_tracks(path: Path) -> list[Track]:
    """IO-адаптер: читає треки з JSON-файлу."""
    return list(json.loads(path.read_text(encoding="utf-8")))


def render(top: list[Track], stats: ListenStats) -> str:
    """Перетворює чистий результат на текст (без друку)."""
    lines = [
        f"{t['id']:>3}  {t['title']:<22} {t['genre']:<5} {t['total_listens']:>9.1f}"
        for t in top
    ]
    return "\n".join(
        [
            *lines,
            f"count={stats['count']:.0f} listens_sum={stats['listens_sum']:.1f} "
            f"avg_listens={stats['avg_listens']:.1f}",
        ]
    )


def parse_args(argv: list[str]) -> argparse.Namespace:
    """Розбір аргументів CLI."""
    parser = argparse.ArgumentParser(description="Рейтинг треків за прослуховуваннями")
    parser.add_argument("--data", type=Path, help="JSON зі списком треків")
    parser.add_argument("--top", type=int, default=3)
    parser.add_argument("--boost-genre", default="rock")
    parser.add_argument("--factor", type=float, default=1.5)
    parser.add_argument("--min-listens", type=float, default=0.0)
    return parser.parse_args(argv)


def main(argv: list[str]) -> None:
    """Точка входу: читання → чистий конвеєр → друк."""
    args = parse_args(argv)
    tracks = load_tracks(args.data) if args.data else SAMPLE
    pipeline = build_chart_pipeline(
        args.top, args.boost_genre, args.factor, args.min_listens
    )
    top = pipeline(tracks)
    print(render(top, reduce_listen_stats(top)))


if __name__ == "__main__":
    main(sys.argv[1:])
```

7. Написала pytest-тести у `tests/test_core.py`: порядок рейтингу Top-N і тай-брейкер за `id` при однаковій кількості прослуховувань, відсутність мутацій вхідного списку, фільтр за порогом, окремі map-кроки, нормалізацію назв і правильність reduce-агрегації (зокрема для порожнього входу):

```python
def test_top_n_order_and_tiebreak(tracks: list[Track]) -> None:
    top = build_chart_pipeline(top_n=3, genre="rock", factor=1.5)(tracks)
    assert [t["id"] for t in top] == [2, 1, 4]
    assert top[1]["total_listens"] == pytest.approx(1200.0)
    assert top[2]["total_listens"] == pytest.approx(1200.0)


def test_tiebreak_by_id_when_listens_equal(tracks: list[Track]) -> None:
    top = build_chart_pipeline(top_n=5, genre="rock", factor=1.5)(tracks)
    keys = [(t["total_listens"], t["id"]) for t in top]
    assert keys == sorted(keys, key=lambda k: (-k[0], k[1]))
    assert [t["id"] for t in top if t["total_listens"] == 1200.0] == [1, 4, 5]


def test_no_mutation(tracks: list[Track]) -> None:
    original = deepcopy(tracks)
    build_chart_pipeline(top_n=5, threshold=500.0)(tracks)
    reduce_listen_stats(with_total_listens()(tracks))
    assert tracks == original


def test_threshold_filter(tracks: list[Track]) -> None:
    top = build_chart_pipeline(top_n=10, genre="none", factor=1.0, threshold=1200.0)(
        tracks
    )
    assert [t["id"] for t in top] == [2, 4, 5]


def test_with_total_listens_and_boost(tracks: list[Track]) -> None:
    totals = list(with_total_listens()(tracks))
    assert [t["total_listens"] for t in totals] == [
        800.0,
        1500.0,
        160.0,
        1200.0,
        1200.0,
    ]
    boosted = list(boost_genre("rock", 2.0)(totals))
    assert [t["total_listens"] for t in boosted] == [
        1600.0,
        1500.0,
        320.0,
        1200.0,
        1200.0,
    ]
    assert "total_listens" not in tracks[0]


def test_normalize_titles(tracks: list[Track]) -> None:
    titles = [t["title"] for t in normalize_titles()(tracks)]
    assert titles[:2] == ["Bohemian Rhapsody", "Shape Of You"]


def test_reduce_listen_stats(tracks: list[Track]) -> None:
    stats = reduce_listen_stats(with_total_listens()(tracks))
    assert stats["count"] == 5
    assert stats["listens_sum"] == pytest.approx(4860.0)
    assert stats["avg_listens"] == pytest.approx(972.0)
    assert reduce_listen_stats([]) == {
        "count": 0.0,
        "listens_sum": 0.0,
        "avg_listens": 0.0,
    }
```

8. Додала тести композиції: результат `compose(...)` збігається з ручним вкладеним ланцюжком викликів, `pipe` і `juxt` працюють як очікується, а базовий

конвеєр з методички дає правильний Top-N, статистику й не мутує вхід:

```python
def test_composition_equals_manual_steps(tracks: list[Track]) -> None:
    manual = take(2)(
        sort_by_listens_desc()(
            min_listens(1000.0)(boost_genre("pop", 1.1)(with_total_listens()(tracks)))
        )
    )
    composed = compose(
        take(2),
        sort_by_listens_desc(),
        min_listens(1000.0),
        boost_genre("pop", 1.1),
        with_total_listens(),
    )
    assert list(composed(tracks)) == list(manual)
    assert [t["id"] for t in manual] == [2, 4]


def test_pipe_and_juxt() -> None:
    assert pipe(3, lambda x: x + 1, lambda x: x * 2) == 8
    assert compose(lambda x: x * 2, lambda x: x + 1)(3) == 8

    def length(xs: list[int]) -> int:
        return len(xs)

    def total(xs: list[int]) -> int:
        return sum(xs)

    assert juxt(length, total)([3, 1, 2]) == (3, 6)


# --- базовий приклад з методички ---


def test_base_pipeline_top_n(records: list[Record]) -> None:
    top = build_pipeline(top_n=2, city="Delhi", factor=1.1)(records)
    assert [r["id"] for r in top] == [1, 3]
    assert top[0]["name"] == "Amit"
    assert top[0]["total"] == pytest.approx(165.0)


def test_base_reduce_stats(records: list[Record]) -> None:
    stats = reduce_stats(build_pipeline(top_n=10)(records))
    assert stats["count"] == 3
    assert stats["sum_total"] == pytest.approx(165.0 + 160.0 + 110.0)
    assert stats["avg_total"] == pytest.approx(435.0 / 3)


def test_base_no_mutation(records: list[Record]) -> None:
    original = deepcopy(records)
    build_pipeline()(records)
    assert records == original
```

9. У `tests/test_properties.py` додала property-based тести на Hypothesis: для випадкових каталогів Top-N завжди є префіксом повного відсортованого

результату, конвеєр не змінює вхід, а статистики узгоджені між собою:

```python
tracks_st = st.lists(
    st.fixed_dictionaries(
        {
            "id": st.integers(min_value=1, max_value=10_000),
            "title": st.text(min_size=1, max_size=12),
            "genre": st.sampled_from(["rock", "pop", "jazz"]),
            "listens": st.lists(st.integers(min_value=0, max_value=10**6), max_size=5),
        }
    ),
    max_size=30,
)


@given(tracks_st, st.integers(min_value=0, max_value=10))
def test_top_n_is_prefix_of_sorted(items: list[Track], n: int) -> None:
    full = build_chart_pipeline(top_n=len(items) + 1)(items)
    top = build_chart_pipeline(top_n=n)(items)
    assert top == full[:n]
    keys = [(-t["total_listens"], t["id"]) for t in full]
    assert keys == sorted(keys)


@given(tracks_st)
def test_pipeline_does_not_mutate(items: list[Track]) -> None:
    original = deepcopy(items)
    build_chart_pipeline(top_n=5, threshold=10.0)(items)
    assert items == original


@given(tracks_st)
def test_stats_are_consistent(items: list[Track]) -> None:
    totals = list(with_total_listens()(items))
    stats = reduce_listen_stats(totals)
    assert stats["count"] == len(items)
    assert abs(stats["listens_sum"] - sum(t["total_listens"] for t in totals)) < 1e-6
    if items:
        assert abs(stats["avg_listens"] * stats["count"] - stats["listens_sum"]) < 1e-6
```

10. Запустила CLI: з п'яти треків із коефіцієнтом 1.5 для жанру rock і порогом 300 відібрано три найпопулярніші; при однаковому значенні 1200.0 порядок визначає `id`. Другий запуск читає дані з `sample_data.json`:

```
C:\Users\maksi\Desktop\labs_functional_for_nastia\lab02> python app.py --top 3 --boost-genre rock --facto
2  Shape Of You           pop      1500.0
  1  Bohemian Rhapsody      rock     1200.0
  4  Blinding Lights        pop      1200.0
count=3 listens_sum=3900.0 avg_listens=1300.0

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab02> python app.py --data sample_data.json --top 2
2  Shape Of You           pop      1500.0
  1  Bohemian Rhapsody      rock     1200.0
count=2 listens_sum=2700.0 avg_listens=1350.0
```

11. Перевірила проєкт автоматичними інструментами: усі 15 тестів pytest проходять, `mypy --strict` не знаходить помилок типів, `ruff` та `black` не мають

зауважень до стилю:

```
C:\Users\maksi\Desktop\labs_functional_for_nastia\lab02> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\maksi\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\maksi\Desktop\labs_functional_for_nastia
configfile: pyproject.toml
plugins: hypothesis-6.170.0, platformdirs-4.12.3
collecting ... collected 15 items

tests\test_core.py::test_top_n_order_and_tiebreak PASSED                 [  6%]
tests\test_core.py::test_tiebreak_by_id_when_listens_equal PASSED        [ 13%]
tests\test_core.py::test_no_mutation PASSED                              [ 20%]
tests\test_core.py::test_threshold_filter PASSED                         [ 26%]
tests\test_core.py::test_with_total_listens_and_boost PASSED             [ 33%]
tests\test_core.py::test_normalize_titles PASSED                         [ 40%]
tests\test_core.py::test_reduce_listen_stats PASSED                      [ 46%]
tests\test_core.py::test_composition_equals_manual_steps PASSED          [ 53%]
tests\test_core.py::test_pipe_and_juxt PASSED                            [ 60%]
tests\test_core.py::test_base_pipeline_top_n PASSED                      [ 66%]
tests\test_core.py::test_base_reduce_stats PASSED                        [ 73%]
tests\test_core.py::test_base_no_mutation PASSED                         [ 80%]
tests\test_properties.py::test_top_n_is_prefix_of_sorted PASSED          [ 86%]
tests\test_properties.py::test_pipeline_does_not_mutate PASSED           [ 93%]
tests\test_properties.py::test_stats_are_consistent PASSED               [100%]

============================== 15 passed in 1.01s ==============================

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab02> python -m mypy .
Success: no issues found in 5 source files

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab02> python -m ruff check .
All checks passed!

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab02> python -m black --check .
All done! ✨ 🍰 ✨
5 files would be left unchanged.
```

Висновок: Під час виконання лабораторної роботи я побудувала функціональний конвеєр перетворень над записами музичного каталогу (варіант 14) з `map`, `filter` і генераторних виразів, зібрала його через власну композицію `compose` з однотипних чистих кроків і виконала reduce-агрегацію з трьома показниками. `lambda` використано у предикаті фільтра, map-кроках і ключі сортування з двома полями; жоден крок не змінює вхідні записи, а I/O ізольовано в `app.py`. Юніт- і property-тести, а також mypy, ruff і black підтвердили коректність рейтингу Top-N, відсутність мутацій і правильність агрегації та композиції.